Term Project Progress Report 3
CSCI 562: Formal Specifications

Scott Johnson
Department of Computer Science
University of North Dakota
s.j.johnson@und.edu

Thomas Perez
Department of Computer Science
University of North Dakota
thomas.perez@und.edu

***Abstract*—This is a project progress report for the term project for CSCI 562: Formal Specifications. This is the third of such reports and will detail an updated class diagram, 2 OCL invariants for methods as per instructions, problems encountered, and updated goals for the final submission.**

1

I. Updated Class Diagram

Figure 1 shows the most recent class diagram for the TCAS-II system onboard an aircraft operated by the fictional "JohnsonAir" company. We specifically chose this name to distinguish between the aircraft under operation (i.e. the one in which the TCAS-II system under UML design, is installed) and other aircraft within the vicinity of this aircraft. There is more information later, regarding the requirements for TCAS-II mandates.

Some notes about this diagram:

- As in the previous report, every `Aircraft` is uniquely identified by its transponder ident. Please see Appendix I for the OCL invariant that ensures this is true.
- Every `Aircraft` has a unique `GeoLocation` that identifies it, as well. We will finish specifying the location invariants (no two Aircraft can have the same location but be different entities) in the final report.
- We added three additional enumerations for the `AdvisoryType`, namely `Clear_Of_Traffic`, `Traffic`, and `Maintain_Vertical_Speed`.

2

- ADS-B/out broadcasts the plane's - ident, geo-location, altitude, horizontal and vertical speed (TAS), and any applicable route changes.
  As a sidebar, ADS-B/in, per se, is not an FAA requirement, however all major schedule flights have them in their cockpit instrument panels. The major airlines will have ADS-B/in-out, ie., both "in", and "out" signals. There's a signal embedded into the full ADS-B/out signal called a squit. This squit is broadcast every second, and is known colloquially as a squitter. (It's akin to the vestigial sideband of an HD television signal, like a sub-carrier frequency, if you will.) This squitter is transmitted to a ground station at the airport, and relays it to ATC (ie., "ground" or "ground control") and used for taxi/ground collision avoidance. This squit only broadcasts - ident, geo-position, altitude (even though the aircraft is on the surface), and speed.
- Class G (uncontrolled) airspace, where FAA restrictions are fairly limited and ADS-B/out is not required, is becoming increasingly rare. For this reason, it is assumed that most all `Aircraft` are equipped with ADS-B/out and thus transmit their GPS coordinates (latitude and longitude).

3

- TCAS-II is mandated for Part-121 (scheduled flights), and Part-135 (on-demand/charter) aircraft operations. TCAS-II requires a Mode-S transponders. For part-135 ops, if/when greater than 30 passengers ("pax"), and turbine engine, TCAS-II is FAA mandated. *This differs from Part-121 flights*. For these flights, MTOW (max' take-off weight) is controlling. Ie., weight is the deciding factor. Whether there are any pax or not, if the Part-121 aircraft is greater than 33,000 pounds, the FAA mandates TCAS-II. However, most passenger (Part-121) airlines will require TCAS-II anyhow.
- We removed the `DataMonitor` object and replaced it with a `ADSB` class that will handle the computational requirements of the system.
- TCAS-II does not provide lateral resolutions. Regarding vertical resolutions, the `targetVerticalSpeed` of an `Advisory` will be positive if the pilot is instructed to climb, negative if the pilot is instructed to descend, or annunciated "level off" if the pilot is instructed to level off. If the pilot is instructed to maintain vertical speed, the `targetVerticalSpeed` will be set to the same value as the `verticalSpeed` of the `Aircraft` at the hand of the pilot flying (PF).
- tau is the "time to close", or effective time to collision, in seconds, assuming both aircraft maintain current climb rate and speed.

II. OCL Invariants

We added specifications detailed in Appendix I, which is a first draft of the class structure, in OCL textual form. This was then validated with the USE graphical interface tool. Following this, we revised it to include invariants, operations, and pre- and post-conditions. The second of these OCL specifications is given in Appendix II.

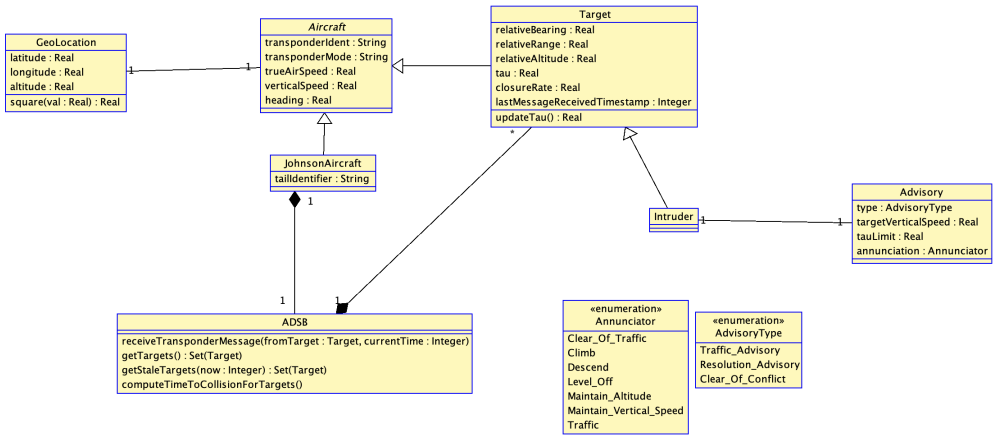

Fig. 1. Updated class diagram for the TCAS-II system

III. Schedule and Progress

We are currently on schedule as per the Gantt chart indicated in progress report 1. We have decided to use Z as our notation for the class hierarchy and general structure, and OCL as the notation for our system logic. The finished versions of both of these will be available as part of the final report.

A. Current Period Goals

The following task(s) have been completed during this review period:

- Class diagram revision (S. Johnson and T. Perez, jointly)
- OCL Initial specification (S. Johnson and T. Perez, jointly)
- Revised OCL specification, with operations and invariants (S. Johnson and T. Perez, jointly)
- Project Progress Report III writing (S. Johnson and T. Perez, jointly)
- Revised Textual Specification (T. Perez, who is our subject matter expert on Pilot Regulations)

B. Next Period Goals

We intend to complete the final report for our next submission period. This will require the following items:

- Finalize UML class diagram (S. Johnson and T. Perez, jointly).
- Finalize any missing logic not present in the OCL specifications (S. Johnson and T. Perez, jointly).
- Document any problems encountered with either `USE` or `fuzz`, so they can be reported in final report (S. Johnson and T. Perez, jointly).
- Translate MS Word template given by Dr. Grant into LaTeXform (S. Johnson).
- Complete writing of final report using Overleaf (S. Johnson and T. Perez, jointly).

C. Issues and Blockers

- When modeling the conversion from a `Target` to an `Intruder`, we can't change the class in OCL. Instead, we needed to model this through the `IntruderAdvisory` association, formally modeling that the system should be in a state where the advises relationship exists and has specific values set. What we do in this case is create a new `Intruder` instance, add it to the set of `Intruders`, and then create an `Advisory` for it. Of course, this will be what happens in *the program that implements this model*. For our purposes, we assert that the states are correct (which is all that can be done in OCL).
- Similar to the previous situation, we're formally going to define `Targets` that have an `Advisory` as `Intruders`, and simply `Targets` if not. This clears up our formal specification by simply adding the tracked aircraft to a different set. In development, the choice can be made to use a set or to subclass it as depicted in the model diagram. We actually debated this quite a bit, and might move to represent this a different way in the class diagram for the final report submission.
- We want to represent that two `Aircraft` can not share the same physical space. We do this by using the 3D Pythagorean Theorem and verifying that it can never be less than 0.001. However, when we perform this operation, we are multiplying altitude (in meters or feet) by latitude and longitude (both in degrees). This would need to be normalized before we proceeded with an implementation, and likely we'd need to add assertions that we're in consistent units. We are aware than normalization to specific units is, historically, a major problem in aerospace engineering, and thus may end up adding invariants for the use of specific units. However, as we've assumed that we're using "microdegrees" for latitude and

longitude, this should be acceptable. In addition, this is a specific safety case, and the program should never be close to this state, so the inclusion of it is mostly a "belt and suspenders" approach to safety (that is, we give a backup method for ensuring safety).

REFERENCES

[1] U.S. Federal Aviation Administration, *14 C.F.R. Part-91 — General Operating and Flight Rules*, Code of Federal Regulations.

[2] U.S. Federal Aviation Administration, *14 C.F.R. Part-121 — Operating Requirements: Domestic, Flag, and Supplemental Operations*, Code of Federal Regulations.

[3] U.S. Federal Aviation Administration, *14 C.F.R. Part-135 — Operating Requirements: Commuter and On-Demand Operations*, Code of Federal Regulations.

[4] U.S. Federal Aviation Administration, *14 C.F.R. Part-71 — Designation of Class A, B, C, D, and E Airspace Areas; Air Traffic Service Routes; and Reporting Points, Code of Federal Regulations*, Code of Federal Regulations.

[5] U.S. Federal Aviation Administration, *14 C.F.R. Part-91.123 — General Operating and Flight Rules*, Code of Federal Regulations.

[6] U.S. Federal Aviation Administration, *14 C.F.R. Part-121.356 — Operating Requirements: Domestic, Flag, and Supplemental Operations*, Code of Federal Regulations.

[7] U.S. Federal Aviation Administration, *14 C.F.R. Part-135.19 and Part-135.180 — Operating Requirements: Commuter and On-Demand Operations*, Code of Federal Regulations.

[8] U.S. Federal Aviation Administration, Traffic Alert and Collision Avoidance System (TCAS) Airborne Equipment, TCAS II, TSO-C119b, Dec. 18, 1998.

APPENDIX I
OCL SPECIFICATION 1

```
model TCAS_II_System

enum AdvisoryType {
    Traffic_Advisory,
    Resolution_Advisory,
    Clear_Of_Conflict
}

enum Annunciator {
    Clear_Of_Traffic,
    Climb,
    Descend,
    Level_Off,
    Maintain_Altitude,
    Maintain_Vertical_Speed,
    Traffic
}

abstract class Aircraft
attributes
    transponderIdent : String
    transponderMode : String
    trueAirSpeed : Real
    verticalSpeed : Real
    heading : Real
end

class GeoLocation
attributes
    latitude : Real
    longitude : Real
    altitude : Real
end

class Target < Aircraft
attributes
    relativeBearing : Real
    relativeRange : Real
    relativeAltitude : Real
    tau : Real
    lastMessageReceivedTimestamp: Integer
end

class JohnsonAircraft < Aircraft
attributes
    tailIdentifier : String
end

class ADSB
end

class Intruder < Target
end

class Advisory
attributes
    type : AdvisoryType
    targetVerticalSpeed : Real
    tauLimit : Real
    annunciation: Annunciator
end

association AircraftLocation between
    Aircraft[1] role contains
    GeoLocation[1] role location
end

composition JohnsonADSB between
    JohnsonAircraft[1] role uses
    ADSB[1] role receives
end
```

```
composition MonitoredIntruders between
    ADSB[1] role monitors
    Target[*] role targets
end

association IntruderAdvisory between
    Intruder[1] role has
    Advisory[1] role advises
end
```

APPENDIX II
OCL SPECIFICATION 2

```
model TCAS_II_System

-- UML Diagram/Structure
enum AdvisoryType {
    Traffic_Advisory,
    Resolution_Advisory,
    Clear_Of_Conflict
}

enum Annunciator {
    Clear_Of_Traffic,
    Climb,
    Descend,
    Level_Off,
    Maintain_Altitude,
    Maintain_Vertical_Speed,
    Traffic
}

abstract class Aircraft
attributes
    transponderIdent : String
    transponderMode : String
    trueAirSpeed : Real
    verticalSpeed : Real
    heading : Real
end

class GeoLocation
attributes
    latitude : Real
    longitude : Real
    altitude : Real

operations
    -- A helper method that we use later
    square(val : Real) : Real = val * val
end

class Target < Aircraft
attributes
    relativeBearing : Real
    relativeRange : Real
    relativeAltitude : Real
    tau : Real
    closureRate : Real
    lastMessageReceivedTimestamp: Integer
operations
   updateTau() : Real = (self.relativeRange / self.closureRate)
end

class JohnsonAircraft < Aircraft
attributes
    tailIdentifier : String
end

class ADSB
operations
    receiveTransponderMessage(fromTarget: Target, currentTime: Integer)
    getTargets() : Set(Target) = self.targets
    getStaleTargets(now : Integer) : Set(Target) =
        self.targets->select(t | now - t.lastMessageReceivedTimestamp > 5)
    computeTimeToCollisionForTargets()
end

class Intruder < Target
end

class Advisory
attributes
    type : AdvisoryType
```

```
    targetVerticalSpeed : Real
    tauLimit : Real
    annunciation: Annunciator
end

association AircraftLocation between
    Aircraft[1] role contains
    GeoLocation[1] role location
end

composition JohnsonADSB between
    JohnsonAircraft[1] role uses
    ADSB[1] role receives
end

composition MonitoredIntruders between
    ADSB[1] role monitors
    Target[*] role targets
end

association IntruderAdvisory between
    Intruder[1] role has
    Advisory[1] role advises
end

-- Constraints and Invariants
constraints

context Aircraft
    inv UniqueTransponderIdent:
        Aircraft.allInstances()->isUnique(transponderIdent)

    inv HeadingWithinReason:
        self.heading >= 0 and self.heading < 360

context ADSB::receiveTransponderMessage(fromTarget: Target, currentTime: Integer)
    post MessageProcessed:
        self.targets->includes(fromTarget) and
        fromTarget.lastMessageReceivedTimestamp = currentTime

context ADSB::computeTimeToCollisionForTargets()
    post TauValuesUpdated:
        self.targets->forAll(t | t.tau >= 0)

    post IssueTrafficAdvisory:
        self.targets->forAll(t |
            (t.tau <= 48 and t.tau >= 20 and t.oclIsTypeOf(Intruder)) implies (
                t.oclAsType(Intruder).advises.type = AdvisoryType::Traffic_Advisory and
                t.oclAsType(Intruder).advises.annunciation = Annunciator::Traffic
            )
        )
    post IssueResolutionAdvisory:
        self.targets->forAll(t |
            (t.tau < 20 and t.oclIsTypeOf(Intruder)) implies (
                t.oclAsType(Intruder).advises.type = AdvisoryType::Resolution_Advisory and
                Set{Annunciator::Climb,
                    Annunciator::Descend,
                    Annunciator::Level_Off,
                    Annunciator::Maintain_Altitude,
                    Annunciator::Maintain_Vertical_Speed}->includes(t.oclAsType(Intruder).advises.annunciation)
            )
        )
    post ClearOfConflictIssued:
        self.targets->forAll(t |
            -- Fix: Apply @pre to the attribute 'tau', not the operation
            (t.tau@pre <= 48 and t.tau@pre >= 20 and t.tau > 48 and t.oclIsTypeOf(Intruder)) implies (
                t.oclAsType(Intruder).advises.type = AdvisoryType::Clear_Of_Conflict and
                t.oclAsType(Intruder).advises.annunciation = Annunciator::Clear_Of_Traffic
            )
        )
    post CleanupResolvedIntruders:
        self.targets->forAll(t |
```

```
            -- Fix: Instead of checking @pre on the type, we check if it HAD an advisory linked
            -- We assume if t.advises was NOT undefined, it was being tracked.
            (t.oclIsTypeOf(Intruder) and
             t.oclAsType(Intruder).advises.type@pre = AdvisoryType::Clear_Of_Conflict) implies
                -- In a formal post-condition, we assert the link is gone
                t.oclAsType(Intruder).advises.oclIsUndefined()
        )

context Intruder
    -- The RA Case
    inv ResolutionAdvisoryRequired:
        self.tau < 20 implies (
            self.advises.type = AdvisoryType::Resolution_Advisory and
            Set{Annunciator::Climb,
                Annunciator::Descend,
                Annunciator::Level_Off,
                Annunciator::Maintain_Altitude,
                Annunciator::Maintain_Vertical_Speed}->includes(self.advises.annunciation)
        )

    -- The TA Case
    inv TrafficAdvisoryRequired:
        (self.tau >= 20 and self.tau <= 48) implies (
            self.advises.type = AdvisoryType::Traffic_Advisory and
            self.advises.annunciation = Annunciator::Traffic
        )

    -- The Recovery Case
    inv ClearOfConflictState:
        self.tau > 48 implies (
            self.advises.type = AdvisoryType::Clear_Of_Conflict and
            self.advises.annunciation = Annunciator::Clear_Of_Traffic
        )

context Aircraft
    inv LocationExists:
        self.location.isDefined()

    -- This uses the 3D Pythagorean theorem to ensure we can _never_ get in a state where
    -- two or more Aircraft are allowed to occupy the same physical space.
    -- NOTE: Altitude is usually in feet or meters, and lat/long are in
    --       degrees. Given that we are using micro-degrees, as specified in the final
    --       report, we can probably assume that this is normalized. However, this should
    --       be verified before the final program is implemented, as we all know that in
    --       aerospace, computation with different units has a history of being a problem.
    inv MinimumSafetySeparation:
        Aircraft.allInstances()->forAll(a1, a2 |
            a1 <> a2 implies (
                a1.location.square(a1.location.latitude - a2.location.latitude) +
                a1.location.square(a1.location.longitude - a2.location.longitude) +
                a1.location.square(a1.location.altitude - a2.location.altitude) > 0.0001
            )
        )
```